

PostgreSQL Server Backup, Decommission and Uninstallation

Production / Enterprise Linux Environments

Document ID	DBA-SOP-PG-DECOM-001
Version	1.0 - Validated Production Runbook
Effective Date	29-Aug-2026 - Saturday
Owner	Database Administration / Platform Engineering
Classification	Internal - Controlled Procedure
Change Requirement	Approved change / maintenance window required
Validation Basis	PostgreSQL 18 official documentation + Ubuntu postgresql-common pg_dropcluster behavior
Target Reference	Ubuntu 26.04 LTS / PostgreSQL 18.6 / cluster 18/main

DESTRUCTIVE-OPERATION WARNING: pg_dropcluster removes the target cluster files and can remove its WAL and tablespace directories. Do not cross the destructive gate until a real restore test has passed, SHA-256 checksums have passed on the off-host copy, tablespace paths have been reviewed, and required approvals are recorded.

1. Purpose

This SOP defines a controlled and auditable procedure to retire a PostgreSQL 18 server/cluster from service. It covers discovery, dependency validation, application quiescence, final logical backups, independent restore testing, preservation of configuration and operational metadata, integrity verification, PostgreSQL cluster removal, package/component uninstallation, residual cleanup, and post-decommission verification.

2. Scope

Use this SOP for PostgreSQL running on Debian/Ubuntu-style Linux systems managed by systemd and postgresql-common. The worked reference values in this document are Ubuntu 26.04 LTS, PostgreSQL 18.6, cluster 18/main, port 5432, and backup root /DBBackups2026. Every site-specific value must be revalidated on the live host before execution. Operating-system or VM retirement is a separate infrastructure activity.

3. Accuracy and Safety Principle

No static runbook can guarantee a safe decommission without validating the live server, application dependencies, replication topology, tablespace paths, backup destination, and restore result. This procedure therefore uses explicit STOP / GO gates. A missing or unexplained result is a NO-GO condition, not an assumption.

4. Roles and Responsibilities

Role	Responsibility	Approval / Evidence
DBA	Inventory, dependency checks, backups, restore validation, PostgreSQL shutdown/removal	Execution log; backup and restore evidence
Application Owner	Stop writes, prevent reconnects, validate outage and confirm retirement	Application sign-off
Change Manager	Authorize maintenance window and destructive phase	Approved change record
Infrastructure / Linux	Package, service, repository and filesystem changes	Server verification
Backup / Storage	Confirm independent copy, destination checksum and retention	Off-host evidence
Security / Compliance (if required)	Validate credential, retention and evidence handling	Policy evidence

5. Preconditions and Change Controls

- ☐ Approved change ticket with implementation, validation, rollback and communication plans.
- ☐ Confirmed maintenance window and application owner approval.
- ☐ Administrative Linux access and PostgreSQL superuser-equivalent backup privileges.
- ☐ Sufficient free space for final dumps and evidence capture.
- ☐ Independent/off-host storage destination available before destructive removal.
- ☐ PostgreSQL version, cluster name, service unit, port, data directory, configuration paths and package source identified.
- ☐ Monitoring, backup, CMDB, DNS/load-balancer, firewall, secrets, automation, ETL, CDC and application dependencies identified.
- ☐ A separate PostgreSQL 18 validation environment is available for a real restore test, or an equivalent independently approved restore-test platform exists.

6. Site-Specific Variables

Item	Reference / Current Case	Execution Value
Host	Revalidate with hostname -f	_____
OS	Ubuntu 26.04 LTS	_____
PostgreSQL server	18.6	_____
Cluster	18/main	_____
Port	5432	_____
Backup root	/DBBackups2026	_____
Server binaries	/usr/lib/postgresql/18/bin	_____
Service	postgresql@18-main.service	_____
Data directory	Revalidate from SHOW data_directory	_____
Config file	Revalidate from SHOW config_file	_____
Off-host backup destination	Organization-approved target	_____
Restore-test host/environment	Separate PostgreSQL 18 target	_____
Change ticket	Required	_____

7. Procedure Overview / Control Gates

- ☐ Phase A - Establish secure execution workspace and identify the target.
- ☐ Phase B - Capture configuration, database inventory, roles, extensions, tablespaces and package inventory.
- ☐ Phase C - Check physical/logical replication, WAL receiver, slots, publications, subscriptions and external dependencies.
- ☐ Phase D - Enter maintenance window, stop application writes and prevent reconnects.
- ☐ Phase E - Re-capture final inventory, take final globals and per-database dumps.
- ☐ Phase F - Validate archive readability, perform a real independent restore test, generate checksums, copy off-host, verify destination checksums.
- ☐ GO/NO-GO GATE - All recovery evidence, dependency evidence and approvals must be GO.
- ☐ Phase G - Stop PostgreSQL and perform last pre-destruction validation.
- ☐ Phase H - Remove cluster, simulate and purge approved packages, inspect residual files.
- ☐ Phase I - Final technical and operational closure.

8. Detailed Execution Procedure

Step 1 - Establish Secure Execution Record and Backup Workspace

```
sudo -i
umask 077

export BACKUP_ROOT=/DBBackups2026

install -d -m 0700 -o postgres -g postgres "$BACKUP_ROOT"
install -d -m 0700 -o postgres -g postgres "$BACKUP_ROOT/decommission_info"
install -d -m 0700 -o postgres -g postgres "$BACKUP_ROOT/config_backup"
install -d -m 0700 -o postgres -g postgres "$BACKUP_ROOT/logs"

date -ls | tee "$BACKUP_ROOT/decommission_info/execution_start.txt"
hostname -f | tee "$BACKUP_ROOT/decommission_info/hostname.txt"
cat /etc/os-release > "$BACKUP_ROOT/decommission_info/os-release.txt"
uname -a > "$BACKUP_ROOT/decommission_info/uname.txt"

df -h "$BACKUP_ROOT"
df -i "$BACKUP_ROOT"
```

- ☐ Change ticket number, executor and maintenance window recorded.
- ☐ Backup filesystem free space and inode availability reviewed.
- ☐ Backup root permissions verified as restricted.

Step 2 - Confirm PostgreSQL Topology, Version, Service and Runtime Paths

```
pg_lsclusters | tee "$BACKUP_ROOT/decommission_info/clusters.txt"

systemctl status postgresql@18-main --no-pager > "$BACKUP_ROOT/decommission_info/postgresql_service.txt" 2>&1 || true

/usr/lib/postgresql/18/bin/postgres --version | tee "$BACKUP_ROOT/decommission_info/postgres_version.txt"
/usr/lib/postgresql/18/bin/pg_dump --version | tee "$BACKUP_ROOT/decommission_info/pg_dump_version.txt"
/usr/lib/postgresql/18/bin/pg_restore --version | tee "$BACKUP_ROOT/decommission_info/pg_restore_version.txt"
/usr/lib/postgresql/18/bin/psql --version | tee "$BACKUP_ROOT/decommission_info/psql_version.txt"

sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT version();
SHOW port;
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
SHOW ident_file;" > "$BACKUP_ROOT/decommission_info/runtime_identity.txt"
```

STOP: If the host, PostgreSQL major version, cluster name, port or data/configuration paths do not match the approved change scope, do not continue.

Step 3 - Capture Package and Component Inventory

```
dpkg -l | grep -Ei 'postgres|libpq|postgis|pgbouncer|pgpool|repmgr' > "$BACKUP_ROOT/decommission_info/components_before_decommission.txt" || true

dpkg -l | awk '
$1=="ii" &&
(
$2 ~ /^postgresql/ ||
$2 ~ /^libpq/ ||
$2 ~ /^pgbouncer/ ||
$2 ~ /^pgpool/ ||
$2 ~ /^repmgr/
)
{print $0}' > "$BACKUP_ROOT/decommission_info/packages_before_removal.txt"
```

```
cat "$BACKUP_ROOT/decommission_info/packages_before_removal.txt"
```

- ☐ Server, client, extension and related support packages reviewed.
- ☐ libpq and other shared client libraries assessed for non-PostgreSQL dependencies.

Step 4 - Capture Configuration and Configuration Source Files

```
cp -a /etc/postgresql/18/main "$BACKUP_ROOT/config_backup/"
cp -a /etc/postgresql-common "$BACKUP_ROOT/config_backup/" 2>/dev/null || true

sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT name, setting, unit, source, sourcefile
FROM pg_settings
WHERE source <> 'default'
ORDER BY name;" > "$BACKUP_ROOT/decommission_info/non_default_settings.txt"

sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT DISTINCT sourcefile
FROM pg_settings
WHERE sourcefile IS NOT NULL
ORDER BY sourcefile;" > "$BACKUP_ROOT/decommission_info/config_source_files.txt"
```

- ☐ postgresql.conf, pg_hba.conf, pg_ident.conf and included configuration files reviewed/preserved.
- ☐ Sensitive configuration and credentials are stored only in approved protected locations.

Step 5 - Capture Dynamic Database Inventory and Sizes

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -Atc "
SELECT datname
FROM pg_database
WHERE datallowconn
AND NOT datistemplate
ORDER BY datname;" > "$BACKUP_ROOT/decommission_info/databases.txt"

cat "$BACKUP_ROOT/decommission_info/databases.txt"

sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT datname,
       pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
WHERE datallowconn
AND NOT datistemplate
ORDER BY pg_database_size(datname) DESC;" > "$BACKUP_ROOT/decommission_info/database_sizes.txt"
```

Reference observation from the supplied environment: user databases alpha, appdb, migrationdb, ramdb and testdb, with postgres used as the maintenance database. The live databases.txt file is authoritative and must not be replaced by a hard-coded list.

STOP: Investigate any unexpected database. The final backup step must use the dynamically re-captured final database list, including postgres if it is connectable and non-template.

Step 6 - Capture Roles, Tablespaces and Extension Inventory

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d postgres -c ""\du+"" > "$BACKUP_ROOT/decommission_info/roles.txt"

sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT oid,
       spcname,
       pg_tablespace_location(oid) AS location
FROM pg_tablespace
ORDER BY spcname;" > "$BACKUP_ROOT/decommission_info/tablespaces.txt"

while IFS= read -r DB; do
[ -n "$DB" ] || continue
```

Controlled document - validate site-specific values before execution

```
echo "===== DATABASE: $DB ====="
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d "$DB" -c ""\dx""
done < "$BACKUP_ROOT/decommission_info/databases.txt" > "$BACKUP_ROOT/decommission_info/extensions.txt"
```

CRITICAL: Review every non-empty custom tablespace path. pg_dropcluster can remove tablespace directories belonging to the cluster. Confirm that no required data for another cluster or application is stored in those paths.

Step 7 - Check Physical Replication, Standby Status and WAL Receiver

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT pg_is_in_recovery();" > "$BACKUP_ROOT/decommission_info/recovery_status.txt"
```

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT * FROM pg_stat_wal_receiver;" > "$BACKUP_ROOT/decommission_info/wal_receiver.txt"
```

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT * FROM pg_stat_replication;" > "$BACKUP_ROOT/decommission_info/replication.txt"
```

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -P pager=off -c "
SELECT slot_name, slot_type, database, active, active_pid, restart_lsn
FROM pg_replication_slots
ORDER BY slot_name;" > "$BACKUP_ROOT/decommission_info/replication_slots.txt"
```

- ☐ Standby/recovery state understood.
- ☐ Upstream WAL receiver status understood.
- ☐ Downstream replicas and replication slots understood.

Step 8 - Check Logical Replication Publications and Subscriptions

```
while IFS= read -r DB; do
[ -n "$DB" ] || continue
echo "===== DATABASE: $DB ====="
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d "$DB" -P pager=off -c "
SELECT subname, subenabled, subslotname
FROM pg_subscription
ORDER BY subname;"
done < "$BACKUP_ROOT/decommission_info/databases.txt" > "$BACKUP_ROOT/decommission_info/subscriptions.txt"
```

```
while IFS= read -r DB; do
[ -n "$DB" ] || continue
echo "===== DATABASE: $DB ====="
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d "$DB" -P pager=off -c "
SELECT pubname, puballtables
FROM pg_publication
ORDER BY pubname;"
done < "$BACKUP_ROOT/decommission_info/databases.txt" > "$BACKUP_ROOT/decommission_info/publications.txt"
```

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d postgres -P pager=off -c "
SHOW archive_mode;
SHOW archive_command;" > "$BACKUP_ROOT/decommission_info/archiving.txt"
```

STOP: Do not decommission until every physical replication, logical replication, CDC, WAL archive, HA/failover and downstream/upstream dependency has an approved retirement or migration disposition.

Step 9 - Identify Active Clients and External Dependencies

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d postgres -P pager=off -c "
SELECT datname, username, client_addr, application_name, state, count(*)
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY datname, username, client_addr, application_name, state
ORDER BY datname, username;" > "$BACKUP_ROOT/decommission_info/client_connections.txt"
```

- ☐ Application connection strings / service accounts identified.

- ☐ Scheduled jobs, ETL, CDC, reporting, monitoring, backup agents, cron/systemd timers and scripts checked.
- ☐ DNS aliases, load balancers, firewall rules, service discovery and CMDB relationships checked.
- ☐ Application owner confirms the database service can be retired.

Step 10 - Enter Maintenance Window and Quiesce Application Writes

Stop or place all dependent applications in maintenance/read-only mode using their approved operating procedures. Prevent automatic reconnects where practical. Stop scheduled writers, ETL, CDC and batch processes. Do not take the final backup while business writes are still possible.

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -d postgres -P pager=off -c "
SELECT pid,
       datname,
       username,
       client_addr,
       application_name,
       state,
       backend_start,
       xact_start,
       query_start
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
ORDER BY datname, username;"
```

- ☐ Application writes stopped.
- ☐ Application reconnects controlled.
- ☐ Unexpected sessions investigated and cleared using approved operational procedures.
- ☐ Application owner confirms maintenance state.

Step 11 - Re-Capture FINAL Database Inventory

```
sudo -u postgres /usr/lib/postgresql/18/bin/psql -X -v ON_ERROR_STOP=1 -d postgres -Atc "
SELECT datname
FROM pg_database
WHERE datallowconn
AND NOT datistemplate
ORDER BY datname;" > "$BACKUP_ROOT/decommission_info/databases_FINAL.txt"
```

```
cat "$BACKUP_ROOT/decommission_info/databases_FINAL.txt"
```

```
diff -u "$BACKUP_ROOT/decommission_info/databases.txt" "$BACKUP_ROOT/decommission_info/databases_FINAL.txt" || true
```

Any difference between the earlier and FINAL database lists must be explained and approved. databases_FINAL.txt is the authoritative final dump list.

Step 12 - Take FINAL Cluster Globals Backup

```
sudo -u postgres /usr/lib/postgresql/18/bin/pg_dumpall -h /var/run/postgresql -p 5432 -U postgres --globals-only --
file="$BACKUP_ROOT/postgresql_globals_FINAL.sql"
```

```
test -s "$BACKUP_ROOT/postgresql_globals_FINAL.sql" && echo "PASS: globals backup exists and is non-empty" || { echo "FAIL: globals backup
missing/empty"; exit 1; }
```

The globals dump preserves cluster-wide objects such as roles and tablespace definitions that individual pg_dump archives do not contain. Treat the file as sensitive because role credential material can be included according to server state and options.

Step 13 - Take FINAL Backups of Every Connectable Non-Template Database

```
sudo -u postgres bash <<'EOF'
set -euo pipefail
umask 077
```

```

BACKUP_ROOT=/DBBackups2026
DBLIST="$BACKUP_ROOT/decommission_info/databases_FINAL.txt"
PGDUMP=/usr/lib/postgresql/18/bin/pg_dump
PGRESTORE=/usr/lib/postgresql/18/bin/pg_restore

while IFS= read -r DB; do
[ -n "$DB" ] || continue

echo "=====
echo "FINAL BACKUP: $DB"
echo "=====

"$PGDUMP" -h /var/run/postgresql -p 5432 -U postgres -Fc -v -f "$BACKUP_ROOT/${DB}_FINAL.dump" "$DB"

test -s "$BACKUP_ROOT/${DB}_FINAL.dump"

"$PGRESTORE" -l "$BACKUP_ROOT/${DB}_FINAL.dump" > "$BACKUP_ROOT/decommission_info/${DB}_archive_contents.txt"

echo "ARCHIVE CATALOG READABLE: $DB"
done < "$DBLIST"

echo "ALL FINAL DATABASE DUMPS COMPLETED."
EOF

```

Important: pg_restore -l only proves that the archive table of contents can be read. It is NOT a full restore test and must not be labeled as complete recoverability validation.

Step 14 - Verify Every Expected Final Dump Exists

```

while IFS= read -r DB; do
[ -n "$DB" ] || continue
FILE="$BACKUP_ROOT/${DB}_FINAL.dump"

if [ ! -s "$FILE" ]; then
echo "FAIL: Missing or empty backup: $FILE"
exit 1
fi

echo "PASS: $FILE"
done < "$BACKUP_ROOT/decommission_info/databases_FINAL.txt"

ls -lh "$BACKUP_ROOT"/*_FINAL.dump

```

□ Every database in databases_FINAL.txt has one non-empty custom-format dump.

Step 15 - Perform a REAL Independent Restore Test

Preferred control: copy the final backup set to a separate PostgreSQL 18 validation host/environment with the required extensions installed. Do not use pg_restore -l as the recovery proof. A successful restore requires pg_restore to reconstruct the database on the validation system.

For each database in databases_FINAL.txt, create an empty test database from template0 and restore with --exit-on-error. Adjust connection parameters only for the approved validation host.

```

# Run on the approved PostgreSQL 18 validation environment.
# Example for one database; repeat dynamically for every database in databases_FINAL.txt.

createdb -T template0 restore_test_alpha

pg_restore --exit-on-error --verbose -d restore_test_alpha alpha_FINAL.dump

psql -X -d restore_test_alpha -c "SELECT current_database();"
psql -X -d restore_test_alpha -c "
SELECT count(*) AS table_count
FROM pg_class
WHERE relkind IN ('r','p');"

```

Controlled document - validate site-specific values before execution

```
psql -X -d restore_test_alpha -c "
SELECT count(*) AS schema_count
FROM pg_namespace;"
```

Perform application/data validation appropriate to each critical database. Capture the validation PostgreSQL version, database name, dump filename, SHA-256, restore result, validation result, date/time and approver.

NO-GO: If any database cannot be restored successfully, or required extension/tablespace dependencies cannot be reproduced and validated, do not remove the source cluster.

Step 16 - Generate SHA-256 Integrity Checksums and Manifest

```
cd "$BACKUP_ROOT" || exit 1

sha256sum *_FINAL.dump postgresql_globals_FINAL.sql > FINAL_SHA256SUMS.txt

sha256sum -c FINAL_SHA256SUMS.txt

ls -lh *_FINAL.dump postgresql_globals_FINAL.sql FINAL_SHA256SUMS.txt > decommission_info/final_backup_manifest.txt

du -sh "$BACKUP_ROOT" > decommission_info/backup_total_size.txt
```

☐ Every checksum reports OK on the source.

☐ Backup sizes reviewed for reasonableness; size alone is not proof of recoverability.

Step 17 - Copy Complete Backup Package to Independent Storage

Copy the complete /DBBackups2026 package to the organization-approved independent storage destination using the approved secure transfer mechanism. The package must include final dumps, globals, FINAL_SHA256SUMS.txt, configuration backup, inventory/evidence files and restore-test evidence.

☐ Off-host copy completed.

☐ Retention period / legal hold / backup ownership recorded.

Step 18 - Verify SHA-256 on the OFF-HOST Destination

```
# Run at the independent destination after transfer.
cd /path/to/copied/DBBackups2026 || exit 1
sha256sum -c FINAL_SHA256SUMS.txt
```

NO-GO: Every listed file must report OK at the destination. A successful copy command without destination checksum verification is not sufficient.

Step 19 - GO / NO-GO Decommission Gate

Gate	Required Evidence	Status
Business approval	Application owner confirms retirement / outage	☐ GO ☐ NO-GO
Final inventory	databases_FINAL.txt reviewed and explained	☐ GO ☐ NO-GO
Backup	Every final database dump + globals successful	☐ GO ☐ NO-GO
Archive readability	pg_restore -l succeeded for every custom archive	☐ GO ☐ NO-GO
Real restore	Independent pg_restore --exit-on-error test passed	☐ GO ☐ NO-GO
Integrity	Source and destination SHA-256 verification passed	☐ GO ☐ NO-GO
Tablespaces	Every custom tablespace path reviewed and approved	☐ GO ☐ NO-GO
Dependencies	Replication/CDC/ETL/clients/archiving/HA cleared	☐ GO ☐ NO-GO
Change approval	Destructive phase explicitly authorized	☐ GO ☐ NO-GO

NO-GO RULE: If any gate is NO-GO, unknown, unexplained, or missing evidence, stop. Do not remove the cluster.

Step 20 - Remove Temporary Backup Authentication Material if Used

If a temporary .pgpass file was created for this maintenance activity, remove it only after backups and restore validation are complete. If no temporary file was created, do not alter pre-existing authentication files without separate approval.


```
# Only if this maintenance created the file:
rm -f /var/lib/postgresql/.pgpass
```

```
test ! -f /var/lib/postgresql/.pgpass && echo "PASS: temporary .pgpass removed"
```

Step 21 - Stop PostgreSQL and Verify It Is Down

```
systemctl stop postgresql@18-main
systemctl stop postgresql || true
```

```
systemctl status postgresql@18-main --no-pager || true
pg_lsclusters || true
```

```
ps -ef | grep '[p]ostgres' || echo "PASS: no postgres processes"
ss -lntp | grep ':5432' || echo "PASS: no TCP listener on 5432"
```

- ☐ Target 18/main cluster shows stopped.
- ☐ No unexplained postgres server process remains.
- ☐ No unexplained listener remains on the configured PostgreSQL port.

Step 22 - Last Pre-Destruction Backup and Tablespace Check

```
cd "$BACKUP_ROOT" || exit 1
sha256sum -c FINAL_SHA256SUMS.txt
```

```
cat "$BACKUP_ROOT/decommission_info/tablespaces.txt"
cat "$BACKUP_ROOT/decommission_info/databases_FINAL.txt"
```

LAST STOP POINT: Confirm the independent copy still exists, restore evidence is attached, source checksums still pass, and every custom tablespace path is approved for removal. The next step is destructive.

Step 23 - Remove the PostgreSQL Cluster

DESTRUCTIVE STEP: pg_dropcluster removes the files belonging to the specified cluster, including data, WAL, configuration and cluster tablespace directories. Execute only after the GO decision.

```
pg_dropcluster --stop 18 main
pg_lsclusters || true
```

- ☐ Target cluster 18/main no longer appears in pg_lsclusters.

Step 24 - Build and Review Exact PostgreSQL Package Removal List

```
dpkg -l | awk '$1=="ii" && $2 ~ /^postgresql/ {print $2}' > /tmp/postgresql_packages_to_remove.txt
```

```
cat /tmp/postgresql_packages_to_remove.txt
```

- ☐ Only packages approved for this decommission are listed.

Step 25 - Simulate Package Purge and Autoremove

```
xargs -r apt-get -s purge < /tmp/postgresql_packages_to_remove.txt
```

```
apt-get -s autoremove --purge
```

Review the complete APT simulation. If unrelated software would be removed, stop and resolve package dependencies before using -y.

Step 26 - Purge Approved PostgreSQL Packages

```
xargs -r apt-get purge -y < /tmp/postgresql_packages_to_remove.txt
```

```
# Re-simulate autoremove after the explicit purge.
apt-get -s autoremove --purge
```

Run only after reviewing the simulation:

```
apt-get autoremove -y --purge
apt-get autoclean
```

☐ Approved PostgreSQL packages removed.

Step 27 - Review Shared and Related Components Separately

```
echo "=== libpq ==="
dpkg -l | awk ' $1=="ii" && $2 ~ /^libpq/ {print}'

echo "=== PgBouncer / Pgpool / repmgr ==="
dpkg -l | awk '
$1=="ii" &&
(
  $2 ~ /^pgbouncer/ ||
  $2 ~ /^pgpool/ ||
  $2 ~ /^repmgr/
)
{print}'
```

Retaining libpq is not a decommission failure when another application requires PostgreSQL client libraries. PgBouncer, Pgpool and repmgr must be explicitly retained or removed based on dependency analysis; do not assume autoremove will make the correct business decision.

Step 28 - Inspect Residual PostgreSQL Directories Before Deletion

```
for DIR in /var/lib/postgresql /etc/postgresql /var/log/postgresql /usr/lib/postgresql /usr/lib/pgsql /var/lib/pgsql
do
  echo
  echo "===== $DIR ====="
  if [ -e "$DIR" ]; then
    find "$DIR" -maxdepth 3 -ls
  else
    echo "Not present"
  fi
done
```

Do not run blanket `rm -rf` commands merely because a path looks PostgreSQL-related. Remove only confirmed residuals that are no longer package-owned, cluster-owned, application-owned or required for recovery. NEVER delete /DBBackups2026 as part of this SOP.

Step 29 - Remove PGDG Repository Configuration Only If Approved

```
ls -l /etc/apt/sources.list.d/*pgdg* 2>/dev/null || true

# Only when the host no longer requires PGDG packages:
rm -f /etc/apt/sources.list.d/pgdg.list
rm -f /etc/apt/sources.list.d/pgdg.sources

apt-get update
systemctl daemon-reload
systemctl reset-failed
```

☐ Repository cleanup follows organizational package-management policy.

Step 30 - Post-Decommission Technical Verification

```
echo "=== PostgreSQL processes ==="
if pgrep -a postgres; then
  echo "FAIL: postgres processes still exist"
else
  echo "PASS: no postgres processes"
fi

echo "=== PostgreSQL clusters ==="
if command -v pg_lsclusters >/dev/null 2>&1; then
  pg_lsclusters
```

Controlled document - validate site-specific values before execution

```

else
  echo "INFO: pg_lsclusters no longer installed"
fi

echo "=== Related services ==="
systemctl list-unit-files | grep -Ei 'postgres|pgbouncer|pgpool|repmgr' || echo "PASS: no related service unit files found"

echo "=== PostgreSQL / related packages ==="
dpkg -l | awk '
$1=="ii" &&
(
  $2 ~ /^postgresql/ ||
  $2 ~ /^pgbouncer/ ||
  $2 ~ /^pgpool/ ||
  $2 ~ /^repmgr/
)
{print}'

echo "=== libpq (informational; may be intentionally retained) ==="
dpkg -l | awk '$1=="ii" && $2 ~ /^libpq/ {print}'

echo "=== PostgreSQL 18 server binary ==="
if [ -x /usr/lib/postgresql/18/bin/postgres ]; then
  echo "FAIL: PostgreSQL 18 server binary still exists"
else
  echo "PASS: PostgreSQL 18 server binary removed"
fi

echo "=== psql (informational) ==="
command -v psql || echo "INFO: psql not found"

echo "=== TCP port 5432 ==="
ss -ltp | grep ':5432' || echo "PASS: nothing listening on TCP 5432"

```

- ☐ Any remaining service, cluster, package, process or listener is explained and approved.

Step 31 - Final Backup Verification and Operational Closure

```

ls -lah /DBBackups2026
cd /DBBackups2026 || exit 1
sha256sum -c FINAL_SHA256SUMS.txt

```

- ☐ Independent/off-host copy still exists and destination checksums remain valid.
- ☐ PostgreSQL monitoring and alerting retired only after technical validation.
- ☐ Scheduled backup/maintenance jobs and automation retired.
- ☐ CMDB/service catalog and ownership records updated.
- ☐ Obsolete DNS/load-balancer/firewall/service-discovery entries removed through approved change processes.
- ☐ Credentials/secrets no longer required are revoked or rotated through approved security procedures.
- ☐ Execution log, restore evidence, manifests, checksums, package inventory and approvals attached to the change record.
- ☐ Application, infrastructure, monitoring, backup and security stakeholders notified.
- ☐ Change closed only after post-change monitoring confirms no unexpected dependency failure.

9. Rollback / Abort Strategy

Rollback is phase-dependent. Before `pg_dropcluster`, abort the change, restart PostgreSQL if it was intentionally stopped, and return applications to the approved pre-change state. After `pg_dropcluster/package purge`, rollback is a rebuild-and-

restore operation: reinstall the required PostgreSQL 18 packages and extension prerequisites, recreate approved tablespace paths, restore cluster globals as appropriate, create target databases, restore the validated archives with pg_restore, reapply configuration, and perform application validation. The independently verified off-host backup and successful restore test are therefore the primary recovery controls for the destructive phase.

- ☐ Rollback owner identified.
- ☐ Required PostgreSQL packages/repositories are obtainable for the retention period or archived according to policy.
- ☐ Restore procedure has been tested against organizational RTO/RPO requirements.

10. Final Completion Checklist

#	Control	Complete
1	Approved change and maintenance window	☐
2	Server/cluster identity confirmed	☐
3	Dynamic database and size inventory captured	☐
4	Roles, tablespaces and extensions captured	☐
5	Configuration and non-default settings captured	☐
6	Package and component inventory captured	☐
7	Standby / WAL receiver / downstream replication checked	☐
8	Replication slots, subscriptions, publications and archiving checked	☐
9	Application, ETL, CDC, automation and external dependencies cleared	☐
10	Application writes quiesced and reconnects controlled	☐
11	FINAL database inventory captured	☐
12	Final globals backup completed	☐
13	Final dump for every database in databases_FINAL.txt completed	☐
14	Every custom archive table of contents readable	☐
15	Independent real restore test passed	☐
16	Source SHA-256 verification passed	☐
17	Independent/off-host backup copy completed	☐
18	Destination SHA-256 verification passed	☐
19	Tablespace paths explicitly reviewed	☐
20	GO/NO-GO approval recorded	☐
21	PostgreSQL service stopped	☐
22	Final pre-destruction checksum/tablespace review passed	☐
23	Cluster 18/main removed	☐
24	APT purge/autoremove simulation reviewed	☐
25	Approved packages purged	☐
26	Shared/related components reviewed separately	☐
27	Residual directories inspected before any manual deletion	☐
28	Repository cleanup completed if in scope	☐
29	No unexplained PostgreSQL processes/listeners/services/clusters remain	☐
30	Final backup package remains valid	☐
31	Monitoring/backups/CMDB/secrets operational cleanup completed	☐
32	Stakeholders notified and evidence attached	☐
33	Change closed	☐

11. Site-Specific Reference - Current PostgreSQL Environment

Item	Observed / Supplied Value
Operating system	Ubuntu 26.04 LTS
PostgreSQL server	18.6
Cluster	18/main
Port	5432 (must be revalidated)
User databases previously observed	alpha, appdb, migrationdb, ramdb, testdb
Maintenance database	postgres
Backup root	/DBBackups2026
PostgreSQL 18 pg_dump	/usr/lib/postgresql/18/bin/pg_dump
PostgreSQL 18 pg_dumpall	/usr/lib/postgresql/18/bin/pg_dumpall
PostgreSQL 18 pg_restore	/usr/lib/postgresql/18/bin/pg_restore
Postgres OS home	/var/lib/postgresql
Final-list rule	Use databases_FINAL.txt; do not hard-code the database list
Recovery proof rule	pg_restore -l is archive readability only; require an actual restore test before destruction

These are reference values derived from the supplied SOP/current case and must be revalidated immediately before execution.

12. Technical Validation Notes

- `pg_dump` exports one database and produces a consistent export even while the database is in use; it does not include cluster-wide roles and tablespaces.
- `pg_dumpall --globals-only` is used to preserve cluster-wide objects such as roles and tablespaces when databases are dumped individually.
- `pg_restore` is the restore utility for custom-format `pg_dump` archives. Listing an archive with `-l` reads its table of contents; a direct restore with `--exit-on-error` provides materially stronger recovery validation.
- Separate `pg_dump` operations use separate snapshots. During a final decommission, stopping business writes avoids the risk of separate databases representing different business points in time.
- `pg_dropcluster` on Debian/Ubuntu `postgresql-common` is destructive and removes files belonging to the target cluster, including data, WAL and tablespace directories as documented by the Ubuntu manpage.

13. Official References

- PostgreSQL 18 - Backup and Restore: <https://www.postgresql.org/docs/18/backup.html>
- PostgreSQL 18 - SQL Dump: <https://www.postgresql.org/docs/18/backup-dump.html>
- PostgreSQL 18 - `pg_dump`: <https://www.postgresql.org/docs/18/app-pgdump.html>
- PostgreSQL 18 - `pg_dumpall`: <https://www.postgresql.org/docs/18/app-pg-dumpall.html>
- PostgreSQL 18 - `pg_restore`: <https://www.postgresql.org/docs/18/app-pgrestore.html>
- PostgreSQL 18 - Monitoring Statistics: <https://www.postgresql.org/docs/18/monitoring-stats.html>
- PostgreSQL 18 - `pg_replication_slots`: <https://www.postgresql.org/docs/18/view-pg-replication-slots.html>
- Ubuntu manpage - `pg_dropcluster`: https://manpages.ubuntu.com/manpages/noble/man1/pg_dropcluster.1.html

14. Approval / Execution Sign-Off

Role	Name	Date/Time	Signature / Ticket Evidence
DBA Executor			
Application Owner			
Infrastructure			
Backup / Storage			
Change Approver			
Security / Compliance (if required)			

Github Repo:

https://github.com/PMSQLDBA/PostgreSQL_Admin_Articles/blob/main/PostgreSQL%2018%20Production%20Decommission%20Runbook.md